AIAC LAB -12

Lab Assignment: Searching Algorithms with AI Assistance

Name : T.Ashwin
Roll No : 2503A52700
Batch No : 24

Task Description #1: Linear Search

Implementation with AI Assistance
Task: Use an AI tool to generate a Python program that implements the Linear Search algorithm.
Prompt : Write a Python program with a function `linear_search(arr, target)` that returns the index or -1, with a docstring giving best, average, and worst-case time complexity. Take the list and the targets as user input. Test with a missing target and an empty list.

#code

```
def linear_search(arr, target):
    """Return index of target, else -1.
    Best case: O(1), Average case: O(n), Worst case: O(n)."""
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1


arr = [int(x) for x in input("Enter list elements separated by spaces: ").split()]
while True:
    t = input("Enter target to search (or 'q' to quit): ")
    if t == 'q':
        break
```

```
    print(f"Index of {t}: {linear_search(arr, int(t))}")
print("Linear Search: O(1) best, O(n) average, O(n) worst.")

#output
```

normal list

Enter list elements separated by spaces: 10 25 30 45 50

Enter target to search (or 'q' to quit): 30

Index of 30: 2

Enter target to search (or 'q' to quit): 10

Index of 10: 0

Enter target to search (or 'q' to quit): 50

Index of 50: 4

Enter target to search (or 'q' to quit): 99

Index of 99: -1

Enter target to search (or 'q' to quit): q

Linear Search: O(1) best, O(n) average, O(n) worst.

Empty list

Enter list elements separated by spaces: (press Enter)

Enter target to search (or 'q' to quit): 5

Index of 5: -1

Enter target to search (or 'q' to quit): q

Linear Search: O(1) best, O(n) average, O(n) worst.

Task Description #2: Binary Search

Implementation with AI Optimization

Task: Use AI to generate an efficient Python program that implements Binary Search on a sorted list.

Prompt: Write a Python program with an iterative `binary_search(arr, target)` for a sorted list, taking the list and targets as user input. Return the index or -1. Explain why the list must be sorted and give the time complexities. Test with the first, last, middle, and a missing element.

#code

```
def binary_search(arr, target):
    """Iterative Binary Search on a sorted list.
    Best case: O(1), Average case: O(log n), Worst case: O(log n)."""
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1


arr = [int(x) for x in input("Enter SORTED list elements separated by spaces: ").split()]
while True:
    t = input("Enter target to search (or 'q' to quit): ")
```

```
    if t == 'q':
        break
    print(f"Index of {t}: {binary_search(arr, int(t))}")
print("The list must be sorted because each step discards half the list.")
print("Binary Search: O(1) best, O(log n) average, O(log n) worst.")
```

#output

Enter SORTED list elements separated by spaces: 5 12 18 24 31 47 56 63 78

Enter target to search (or 'q' to quit): 5

Index of 5: 0

Enter target to search (or 'q' to quit): 78

Index of 78: 8

Enter target to search (or 'q' to quit): 31

Index of 31: 4

Enter target to search (or 'q' to quit): 40

Index of 40: -1

Enter target to search (or 'q' to quit): q

The list must be sorted because each step discards half the list.

Binary Search: O(1) best, O(log n) average, O(log n) worst.

Task Description #3: Student Record Search System

Scenario: A university maintains student records containing roll numbers, names, and marks. Faculty members need to search for student records using roll numbers.

Prompt: Write a Python program that takes student records (roll number, name, marks) as user input. Implement Linear Search and Binary Search by roll number. Search the

unsorted and sorted lists and compare the results. Print a comparison table of algorithm, input requirement, and time complexity.

#code

```
def linear_search(recs, roll):
    for i in range(len(recs)):
        if recs[i]["roll"] == roll:
            return i
    return -1
def binary_search(recs, roll):
    low, high = 0, len(recs) - 1
    while low <= high:
        mid = (low + high) // 2
        if recs[mid]["roll"] == roll:
            return mid
        elif recs[mid]["roll"] < roll:
            low = mid + 1
        else:
            high = mid - 1
    return -1
n = int(input("Enter number of students: "))
records = []
for i in range(n):
    r, name, m = input(f"Student {i+1} (roll name marks): ").split()
    records.append({"roll": int(r), "name": name, "marks": float(m)})
sorted_records = sorted(records, key=lambda s: s["roll"])
while True:
```

```
    t = input("Enter roll number to search (or 'q' to quit): ")
    if t == 'q':
        break
    t = int(t)
    print(f" Unsorted: Linear -> {linear_search(records, t)}, Binary -> {binary_search(records,t)}")
    print(f" Sorted:   Linear -> {linear_search(sorted_records, t)}, Binar>{binary_search(sorted_records,t)}")
print("\nAlgorithm | Input requirement | Time complexity")
print("Linear    | Any order         | O(n)")
print("Binary    | Must be sorted    | O(log n)")
```

#output

```
Enter number of students: 5
Enter number of students: 5
Student 2 (roll name marks): 101 Ravi 75
tudent 3 (roll name marks): 110 Meena 92
Student 4 (roll name marks): 103 Kiran 67
Student 5 (roll name marks): 108 Sana 81
Enter roll number to search (or 'q' to quit): 101
Unsorted: Linear -> 1, Binary -> -1
Sorted: Linear -> 0, Binary -> 0
Enter roll number to search (or 'q' to quit): 108
Unsorted: Linear -> 4, Binary -> -1
Sorted: Linear -> 3, Binary -> 3
Enter roll number to search (or 'q' to quit): 999
```

Unsorted: Linear -> -1, Binary -> -1

Sorted: Linear -> -1, Binary -> -1

Binary Search gives -1 on the unsorted list even for roll numbers that exist. That is the key point of this task.

Task Description #4: Library Book Search System

Scenario: A college library maintains a list of book identification numbers. Students and librarians need to find books quickly using their book IDs.

Prompt: Write a Python program that takes book IDs as user input and searches for a book ID using Linear Search and Binary Search (on the sorted list). Both functions should return the index and the number of comparisons. Test with an ID that exists and one that doesn't, and explain which algorithm suits a large sorted collection.

#code

```
def linear_search(ids, target):
    comps = 0
    for i in range(len(ids)):
        comps += 1
        if ids[i] == target:
            return i, comps
    return -1, comps


def binary_search(ids, target):
    low, high, comps = 0, len(ids) - 1, 0
    while low <= high:
        mid = (low + high) // 2
        comps += 1
        if ids[mid] == target:
            return mid, comps
```

```
        elif ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comps

ids = sorted(int(x) for x in input("Enter book IDs separated by spaces: ").split())
while True:
    t = input("Enter book ID to search (or 'q' to quit): ")
```

Enter book ID to search (or 'q' to quit): q

Conclusion: Binary Search needs far fewer comparisons, so it suits large sorted collections.

Task Description #5: Searching Algorithm Comparison and Optimization

Scenario: An online shopping platform maintains product IDs for thousands of products. The system must locate a product quickly when a customer enters a product ID.

prompt: Write a Python program that takes input sizes from the user and generates sorted product IDs of each size. Implement Linear and Binary Search with comparison counts. For each size, test the first, middle, last, and a missing element. Print execution time and comparisons in a table, then state which algorithm suits sorted product IDs.

#code

```
import time
def linear_search(arr, target):
    comps = 0
    for i in range(len(arr)):
        comps += 1
        if arr[i] == target:
            return i,comps
    return -1, comps
def binary_search(arr, target):
    low, high, comps = 0, len(arr) - 1, 0
    while low <= high:
        mid = (low + high) // 2
        comps += 1
        if arr[mid] == target:
```

```
            return mid,comps
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
            return -1, comps
sizes = [int(x) for x in input("Enter input sizes separated by spaces: ").split()]
print(f"{'Size':<8}{'Target':<9}{'Algo':<8}{'Comps':<8}{'Time(ms)'}")
for n in sizes:
    ids = list(range(1, n + 1))
    for label, t in [("first", ids[0]), ("middle", ids[n // 2]), ("last", ids[-1]), ("missing", -1)]:
        for name, fn in (("Linear", linear_search), ("Binary", binary_search)):
            s = time.perf_counter()
            _, c = fn(ids,t)
            print(f"{n:<8}{label:<9}{name:<8}{c:<8}{(time.perf_counter()-s)*1000:.4f}")
print("Linear: O(n). Binary: O(log n), needs sorted data -> best for sorted product IDs.")
```

#output

```
Enter input sizes separated by spaces: 1000 10000 100000
Size    Target   Algo    Comps   Time(ms)
1000    first    Linear  1       0.0086
1000    first    Binary  9       0.0070
1000    middle   Linear  501     0.0326
1000    middle   Binary  9       0.0045
1000    last     Linear  1000    0.0331
```

1000 last Binary 10 0.0042

1000 missing Linear 1000 0.0514

1000 missing Binary 9 0.0046

10000 first Linear 1 0.0037

10000 first Binary 13 0.0037

10000 middle Linear 5001 0.1630

10000 middle Binary 13 0.0032

10000 last Linear 10000 0.3372

10000 last Binary 14 0.0026

10000 missing Linear 10000 0.3392

10000 missing Binary 13 0.0039

100000 first Linear 1 0.0085

100000 first Binary 16 0.0072

100000 middle Linear 50001 2.0594

100000 middle Binary 16 0.0097

100000 last Linear 100000 3.5413

100000 last Binary 17 0.0104

100000 missing Linear 100000 3.4256

100000 missing Binary 16 0.0090

Linear: O(n). Binary: O(log n), needs sorted data -> best for sorted product IDs.